

Explainable XXXClassification of Payment Difficulty

Master in Artificial Intelligence
Universidad Politecnica de Madrid

2026

Abstract

This report documents the first phase of the Tema 2 practical work in Explainable Artificial Intelligence. The accompanying notebook builds a transparent classification pipeline for predicting whether a customer will experience payment difficulty in the following month. It combines categorical encoding, a stratified validation split, an interpretable classification tree, conventional validation metrics, and global explanations.

1 Introduction

The task is a binary classification problem based on customer credit and repayment information. The training data contain the target variable `payment_difficulty_next_month`; the test data contain the same predictors without that target. The objective is both predictive and explanatory: the model should produce test predictions while allowing its behavior to be inspected.

The complete implementation is provided in `code/phase-1.ipynb`. Data files are kept in `data/`, and the notebook uses paths relative to its new location under `code/`.

2 Data and preprocessing

The notebook defines the feature roles explicitly rather than selecting columns implicitly. The identifier and target are excluded from the predictor matrix, while demographic, credit-limit, billing, payment, and repayment variables are used as predictors. Sex is retained as an audit attribute as well as a predictor, which makes later fairness-oriented analysis possible.

Categorical variables are `sex`, `education`, `marital_status`, and `repayment_assistance_plan`. They are transformed with one-hot encoding using `handle_unknown="ignore"`, so categories that appear only in the test split do not cause prediction failures. Numerical variables are concatenated with the encoded categorical variables without additional scaling.

The labeled training data are split into training and validation partitions with a 70/30 ratio, a fixed random seed of 411, and stratification by the target. The unlabeled test data remain untouched until the final prediction stage. This procedure gives a reproducible validation estimate while preserving the test set for submission.

3 Model and explainability

The classifier is `ClassificationTree` from the `interpret` package. A decision tree is suitable for this phase because its decision structure can be inspected directly and because the package exposes global explanations of the fitted model [Nori et al., 2019]. The notebook also displays a class histogram for the training data using the same explainability toolkit.

After fitting, the model predicts the validation partition. The notebook reports accuracy, balanced accuracy, and a full classification report. Balanced accuracy is included because it

averages recall across classes and is therefore more informative than accuracy when the classes are not equally represented [Brodersen et al., 2010].

A global explanation is generated with `tree.explain_global`. This view supports inspection of the features and structures used by the tree. The explanation should be interpreted together with validation performance: a visually simple model is useful only if its predictive behavior is also adequate for the intended task.

4 Test predictions

Once validation is complete, the fitted tree predicts the unlabeled test set. The notebook copies the original test predictors, appends the `predicted_class` column, and writes the result to `data/test_predictions.csv`. It also plots the proportions of the predicted classes. Keeping the original predictors beside the prediction makes the generated file easier to audit and debug.

5 Limitations and future work

This phase uses a single hold-out split and one tree configuration. The reported validation metrics may therefore vary if the split changes, and they do not quantify uncertainty across samples. Future work should compare several seeds or use cross-validation, tune the tree configuration using a validation protocol, and report class-specific error costs if the application makes false negatives and false positives unequal.

The notebook already preserves sex as an audit attribute, but it does not yet calculate group-wise performance or fairness measures. A subsequent phase should compare error rates across relevant groups, inspect feature importance stability, and use local explanations for representative correct and incorrect predictions. Any such analysis should be performed without using the test labels.

6 Reproducibility

The repository layout is:

```
code/phase-1.ipynb
code/requirements.txt
data/train.csv
data/test.csv
data/test_predictions.csv
report/main.tex
report/references.bib
report/figures/
```

Install the dependencies from `code/requirements.txt`, open the notebook from `code/`, and execute its cells in order. The report can be compiled with a LaTeX installation using `main.tex` and `references.bib`.

References

Kay Henning Brodersen, Cheng Soon Ong, Klaas E. Stephan, and Joachim M. Buhmann. The balanced accuracy and its posterior distribution. *2010 20th International Conference on Pattern Recognition*, pages 3121–3124, 2010. doi: 10.1109/ICPR.2010.764.

Harsha Nori, Samuel Jenkins, Paul Koch, and Rich Caruana. Interpretml: A unified framework for machine learning interpretability. In *Proceedings of the 2019 Workshop on Human-Centric Machine Learning at NeurIPS*, 2019. URL <https://github.com/interpretml/interpret>.